

Simplex FastICA

Nimal Sreekumar, Praful Keasavdas , Venkata Chanakya

The Challenge: Blind Source Separation (BSS)

What is BSS?

BSS is the problem of separating a set of mixed signals back into their original, individual sources, without knowing how they were mixed.

Original Sources (S)

- Voice 1
- Voice 2
- Music

Mixed Signals (X)

- Mic 1: Mix of all three
- Mic 2: Different mix of all three

Goal: Recover the original sources (S) from the mixed signals (X).

The Solution: FastICA Algorithm

What is FastICA (FICA)?

A popular and efficient algorithm for Independent Component Analysis (ICA), which is a powerful technique to solve the BSS problem.

Key Idea: FICA finds an "unmixing" matrix $\mathbf{W}$ that maximizes the **non-Gaussianity** of the separated signals.

- Most real-world signals (speech, EEG) are non-Gaussian.
- Random noise is Gaussian.
- By maximizing non-Gaussianity, we find the interesting, independent signals.

The Problem: The standard FICA algorithm is computationally intensive, making it difficult for low-power, real-time applications (e.g., portable medical devices).

The Conventional FICA Iteration Loop

To find each component of the unmixing matrix ($\mathbf{w}$), FICA uses an iterative loop:

- 1 **Vector Update:** Adjust the vector $\mathbf{w}$ to increase non-Gaussianity.

$$w_{i,k}^{\text{new}} = E \left[z_{k,j} \left(\sum_{p=1}^n z_{p,j} w_{i,p} \right)^3 \right] - 3w_{i,k}$$

- 2 **Orthogonalization:** Ensure the new vector is perpendicular to all previously found vectors (using Gram-Schmidt Orthogonalization).

$$w_i^{\text{new}} = w_i^{\text{new}} - \sum_{j=1}^{i-1} (w_i \cdot w_j^c) w_j^c$$

- 3 **Normalization:** Scale the vector's length to 1.

$$w_i^{\text{new}} = \frac{w_i^{\text{new}}}{\|w_i^{\text{new}}\|}$$

- 4 **Convergence Check:** Repeat until the vector stops changing.

The Core Idea: Exploiting Redundancy

The Key Insight

Once the first $n-1$ orthogonal vectors have been found, the direction of the final n -th vector is already geometrically determined. There is only one possible direction left!

In 3D Space:

- After finding w_1 (x-axis)...
- And finding w_2 (y-axis)...
- The direction of w_3 (z-axis) is fixed!

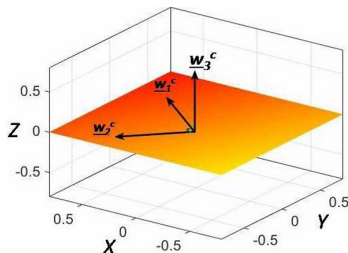


Fig. 1. Results for independent source separation for 3D conventional FICA using MATLAB.

3D

Orthogonal Vectors

Conclusion: The computationally expensive iterative search is unnecessary for the final vector, w_n .

The "Simplex" Shortcut

For the final vector, w_n , Simplex FICA **eliminates the entire iterative loop**.

What is Removed for the n-th Vector?

- X The iterative Vector Update step.
- X The Normalization step.
- X The Convergence Check step.

What is Kept?

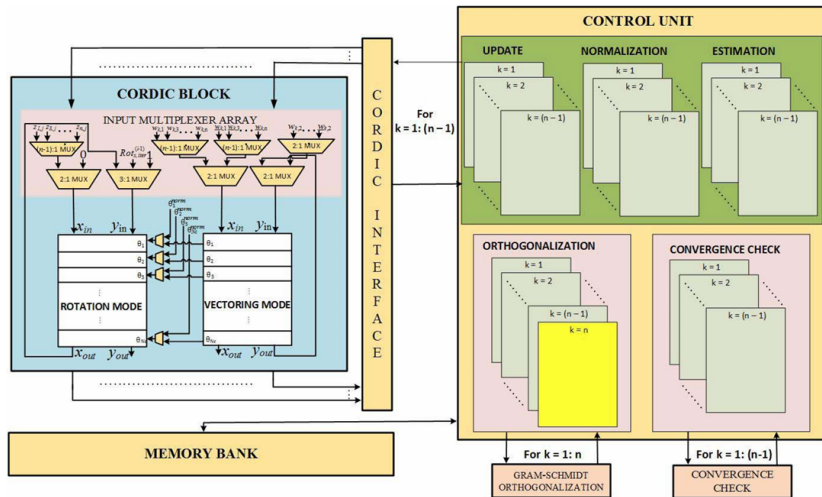
A single, direct calculation:

- ✓ One Gram-Schmidt Orthogonalization (GSO) step to make w_n perpendicular to all other vectors.

This replaces thousands of clock cycles with a single, fast computation.

System Architecture Overview

The hardware is designed to be efficient and reusable.



Data Flow: How it Works

The Control Unit directs the data flow based on which vector is being computed.

For the first $n-1$ vectors (w_1 to w_{n-1}):

The data follows the **full iterative loop**:

- 1 Vector goes to **GSO Block**.
- 2 Then to **CORDIC Block** for Update.
- 3 Back to **CORDIC Block** for Normalization.
- 4 Result is checked in the **Convergence Block**.
- 5 Loop repeats until converged.

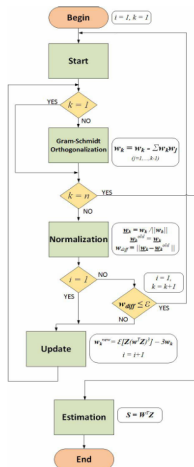
For the final vector (w_n):

The data follows the **accelerated "Simplex" path**:

- 1 Initial vector goes directly to the **GSO Block**.
- 2 **Done.** The result is the final vector.

Algorithmic State Machine (ASM) Flowchart

This flowchart illustrates the complete implementation flow, highlighting the separate paths for vectors $w_1..w_{n-1}$ and the final vector w_n .



What is Simplex FICA?

An accelerated and low-complexity design methodology for nD FastICA that exploits algorithmic redundancy.

Key Contributions:

- **Significant Speedup:** By replacing the iterative loop for the final vector with a single direct calculation, it drastically reduces computation time.
- **Low Complexity:** It achieves this speedup **without adding any extra hardware**, reusing the existing architectural blocks.
- **Practicality:** The design is ideal for resource-constrained, real-time systems such as portable healthcare monitors and IoT devices.